Entropy-Based Feedback Control for LLM Sampling: Mathematical Formalization

1 Observation Model

At each generation step k, the model produces a logit vector over the full vocabulary. We extract the top-K logits $\mathbf{z}_k \in \mathbb{R}^K$ (with $K = 64$) and compute the Shannon entropy of the induced probability distribution:

$$H_k = -\sum_{i=1}^{K} p_i \ln p_i, \qquad p_i = \frac{e^{z_i}}{\sum_{j=1}^{K} e^{z_j}} \tag{1}$$

Entropy is measured in nats (natural logarithm).

Listing 1: Shannon entropy over top-K softmaxed logits (run.py:59–67)

```
def entropy(logits: np.ndarray, top_k: int = 64) -> float:
    idx = np.argpartition(logits, -top_k)[-top_k:]
    v = logits[idx]
    v = v - v.max()           # log-sum-exp stability trick
    e = np.exp(v)
    p = e / e.sum()            # softmax
    mask = p > 0
    return -float(np.sum(p[mask] * np.log(p[mask])))
```

The subtraction `v = v - v.max()` is the log-sum-exp trick for numerical stability. It does not affect the softmax output, since for any constant c:

$$\frac{e^{z_i - c}}{\sum_j e^{z_j - c}} = \frac{e^{z_i}}{\sum_j e^{z_j}} \tag{2}$$

2 Error Signal

The raw error is the deviation from the target entropy setpoint H^*. We apply a sigmoidal dampener via tanh to soft-limit large errors to ± 1:

$$e_k^{\text{raw}} = H^* - H_k \tag{3}$$

$$e_k = \tanh\!\left(\alpha \cdot e_k^{\text{raw}}\right) \tag{4}$$

Sign convention: $e_k > 0$ when entropy is below target (model is too confident); $e_k < 0$ when entropy is above target (model is too uncertain).

Listing 2: Error signal with tanh dampener (run.py:150–153)

```
e_raw = self.H_target - H
e = float(np.tanh(self.sigmoid_alpha * e_raw))
```

3 State Vector

The controller maintains a 4th-order state vector composed of the integral, error, first difference, and second difference of the dampened error signal:

$$\mathbf{x}_k = \begin{bmatrix} I_k \\ e_k \\ \Delta e_k \\ \Delta^2 e_k \end{bmatrix} \tag{5}$$

where:

$$I_k = \text{clip}\Big(I_{k-1} + \tfrac{1}{2}(e_k + e_{k-1}),\ -5,\ 5\Big) \tag{6}$$

$$\Delta e_k = e_k - e_{k-1} \tag{7}$$

$$\Delta^2 e_k = \Delta e_k - \Delta e_{k-1} \tag{8}$$

The integral uses the trapezoidal rule. The first and second differences are discrete finite differences at unit time step ($\Delta t = 1$ token).

Listing 3: State vector update (run.py:155–168)

```
# Finite differences (on dampened signal)
d_e = e - self._prev_error
d2_e = d_e - self._prev_d_error

# Integral (trapezoidal, with anti-windup clamp)
integral = self._x[0]
integral += 0.5 * (e + self._prev_error)
integral = max(-5.0, min(5.0, integral))

# Pack state vector: [integral, error, d_error, d2_error]
self._x[0] = integral
self._x[1] = e
self._x[2] = d_e
self._x[3] = d2_e
```

4 Control Law: Position Form

Three actuators are driven by the state vector via independent gain vectors $\mathbf{K}_M, \mathbf{K}_P, \mathbf{K}_F \in \mathbb{R}^4$:

$$\Delta M_k = -\mathbf{K}_M \cdot \mathbf{x}_k \tag{9}$$

$$\Delta P_k = \ \ \mathbf{K}_P \cdot \mathbf{x}_k \tag{10}$$

$$\Delta F_k = -\mathbf{K}_F \cdot \mathbf{x}_k \tag{11}$$

The sign conventions encode the actuator semantics: positive error (too confident) lowers Min-P to widen the sampling tail, raises Top-P to include more probability mass, and lowers frequency penalty to permit repetition. Actuators are applied as offsets from base values with saturation:

$$M_k = \max\big(0,\ M_0 + \Delta M_k\big) \tag{12}$$

$$P_k = \mathrm{clip}\big(P_0 + \Delta P_k,\ 0,\ 1\big) \tag{13}$$

$$F_k = \max\big(0,\ F_0 + \Delta F_k\big) \tag{14}$$

Listing 4: Actuation in position form (run.py:171–183)

```
# Actuator deltas via dot product with state vector
dM = -float(self.K_M @ self._x)
dP =  float(self.K_P @ self._x)
dF = -float(self.K_F @ self._x)

# Apply as offset from base, with saturation
M = max(0.0, self.M_base + dM)
P = max(0.0, min(1.0, self.P_base + dP))
F = max(0.0, self.F_base + dF)
```

5 Control Law: Velocity Form

In the velocity form, actuator outputs accumulate from step to step rather than resetting to a base offset. The Δ computation is unchanged; only the accumulation differs:

$$M_k = \max\big(0,\ M_{k-1} + \Delta M_k\big) \tag{15}$$

$$P_k = \mathrm{clip}\big(P_{k-1} + \Delta P_k,\ 0,\ 1\big) \tag{16}$$

$$F_k = \max\big(0,\ F_{k-1} + \Delta F_k\big) \tag{17}$$

with initial conditions $M_0 = M_{\text{base}}$, $P_0 = P_{\text{base}}$, $F_0 = F_{\text{base}}$. The velocity form permits the actuator to sustain corrections over extended generation sequences without requiring persistent error to maintain the offset.

Listing 5: Actuation in velocity form (modified run.py:171–183)

```
# Actuator deltas (same as position form)
dM = -float(self.K_M @ self._x)
dP =  float(self.K_P @ self._x)
dF = -float(self.K_F @ self._x)

# Accumulate from previous step, with saturation
self._M = max(0.0, self._M + dM)
self._P = max(0.0, min(1.0, self._P + dP))
self._F = max(0.0, self._F + dF)

M, P, F = self._M, self._P, self._F
```

6 Quantum Early Warning Signal (QEWS)

At each step k, the top-K raw logit vector is L_2-normalized to obtain a state vector on the unit sphere:

$$\boldsymbol{\psi}_k = \frac{\mathbf{z}_k}{\|\mathbf{z}_k\|_2} \tag{18}$$

Note that $\boldsymbol{\psi}_k$ is constructed from raw logits, *not* softmaxed probabilities. This preserves magnitude relationships that softmax compresses.

Over a rolling window of width W (default $W = 12$), we construct the density operator:

$$\boldsymbol{\rho}_k = \frac{1}{W} \sum_{i=k-W+1}^{k} \boldsymbol{\psi}_i \boldsymbol{\psi}_i^\top \tag{19}$$

$\boldsymbol{\rho}_k \in \mathbb{R}^{K \times K}$ is positive semidefinite with unit trace by construction. Let $\{\lambda_j\}_{j=1}^{K}$ denote its eigenvalues. The von Neumann entropy of the density operator is:

$$S_k = -\sum_{j=1}^{K} \lambda_j \ln \lambda_j \tag{20}$$

with the convention $0 \ln 0 = 0$. We maintain an exponential moving average of S_k and define the QEWS signal as the deviation:

$$\mu_k = (1 - \alpha_{\text{ema}})\, \mu_{k-1} + \alpha_{\text{ema}}\, S_k, \qquad \alpha_{\text{ema}} = 0.05 \tag{21}$$

$$\text{QEWS}_k = S_k - \mu_k \tag{22}$$

Listing 6: QEWS density operator and von Neumann entropy (run.py:217–255)

```python
# Extract top-K logits and normalize to unit vector
idx = np.argpartition(logits, -self.K)[-self.K:]
v = torch.tensor(logits[idx], device=self.device,
                 dtype=torch.float32)
norm = torch.linalg.norm(v)
if norm > 1e-12:
    v = v / norm

# Insert into circular buffer
self._buf[self._ptr] = v
self._ptr = (self._ptr + 1) % self.W
self._count += 1

# Burn-in: need full window
if self._count < self.W:
    return 0.0, 0.0, 0.0

# Density operator: rho = (1/W) * sum(|psi><psi|)
rho = (self._buf.T @ self._buf) / self.W

# Von Neumann entropy: S = -sum(lambda * log(lambda))
eigvals = torch.linalg.eigvalsh(rho)
eigvals = eigvals.clamp(min=1e-12)
S_raw = -float(torch.sum(eigvals * torch.log(eigvals)))
```

```python

# Exponential moving average baseline
if not self._warmed_up:
    self._mu_S = S_raw
    self._warmed_up = True
else:
    self._mu_S += self.ema_alpha * (S_raw - self._mu_S)

signal = S_raw - self._mu_S
return S_raw, signal, self._mu_S
```

7 Hybrid Mode

In hybrid mode, two independent 4th-order controllers operate in parallel: one driven by Shannon entropy error $e_k^H = H^* - H_k$ and one driven by QEWS error $e_k^Q = Q^* - \text{QEWS}_k$. Their actuator deltas are combined via weighted sum:

$$\Delta M_k^{\text{hybrid}} = w_H \Delta M_k^H + w_Q \Delta M_k^Q \tag{23}$$

$$\Delta P_k^{\text{hybrid}} = w_H \Delta P_k^H + w_Q \Delta P_k^Q \tag{24}$$

$$\Delta F_k^{\text{hybrid}} = w_H \Delta F_k^H + w_Q \Delta F_k^Q \tag{25}$$

Setting $w_Q = 0$ recovers pure entropy control; setting $w_H = 0$ recovers replace mode.

Listing 7: Hybrid mode actuator combination (run.py:539–549)

```python
M_h, P_h, F_h, dM_h, dP_h, dF_h = controller.step(H)
M_q, P_q, F_q, dM_q, dP_q, dF_q = qews_controller.step(qews_sig)
dM = w_H * dM_h + w_Q * dM_q
dP = w_H * dP_h + w_Q * dP_q
dF = w_H * dF_h + w_Q * dF_q
M = max(0.0, controller.M_base + dM)
P = max(0.0, min(1.0, controller.P_base + dP))
F = max(0.0, controller.F_base + dF)
```

8 Actuator Semantics

The three actuators control distinct aspects of the sampling distribution:

Min-P (M). A token with probability p_i is retained if $p_i \geq M \cdot p_{\max}$, where $p_{\max}$ is the probability of the highest-ranked token. Min-P adapts to model confidence: high-confidence distributions (large $p_{\max}$) produce a high floor, retaining few tokens; low-confidence distributions produce a low floor, retaining many.

Top-P (P). Nucleus sampling: tokens are retained in descending probability order until their cumulative probability reaches P. All remaining tokens are zeroed.

Frequency Penalty (F). Logits of previously generated tokens are reduced by F times their occurrence count before sampling. This discourages repetition.